SAÚDE CERTA

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas – TADS
Instituição: Instituto Federal de Educação, Ciência e Tecnologia do Pará – IFPA
Disciplina: Desenvolvimento de Aplicações Multicamadas
Turma: C795NL
Docente: Márcio Goes do Nascimento

Equipe:

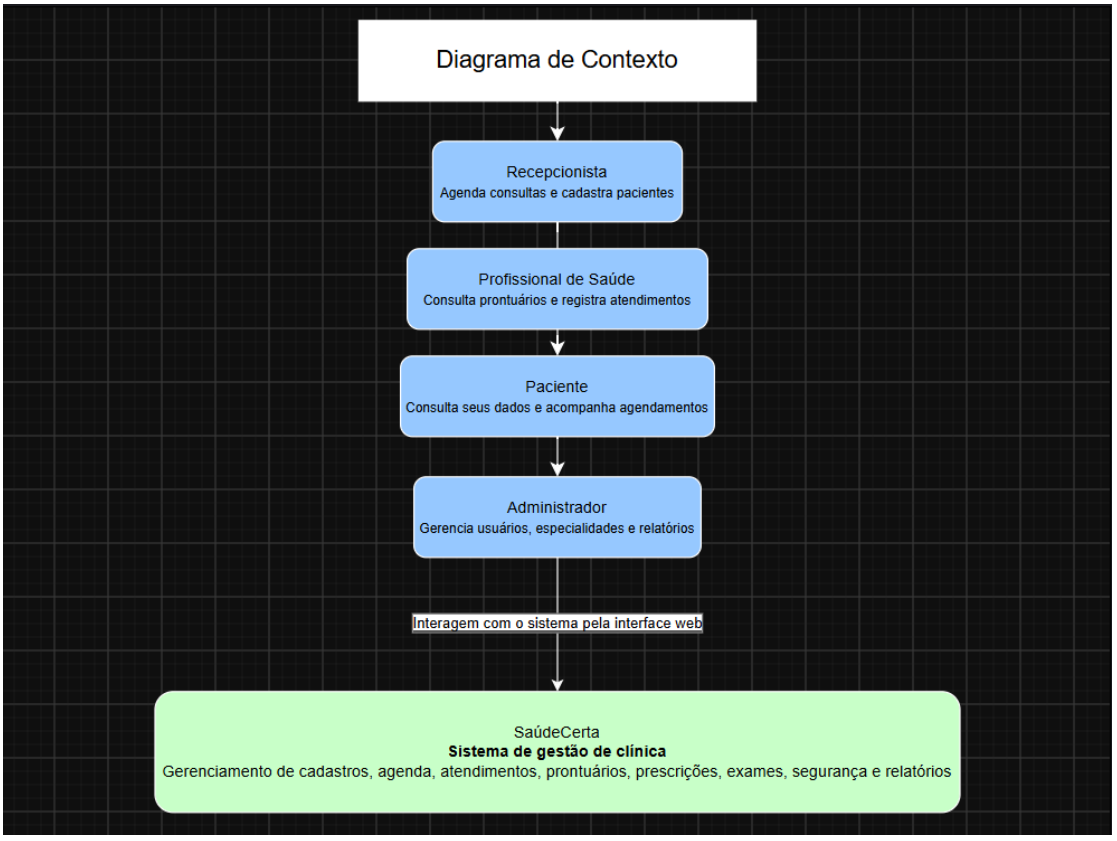
- Júlia Rodrigues
- Marielli Macedo
- Ludmila Johnston
- Ronaldo Ângelo

Repositório GitHub:
https://github.com/Ludmilajohnston/SaudeCerta

1. Diagramas C4

Os diagramas C4 apresentam a arquitetura do sistema em diferentes níveis de detalhamento. Foram elaborados os diagramas de Contexto, Contêiner e Componente.

1.1 Diagrama C4 – Contexto



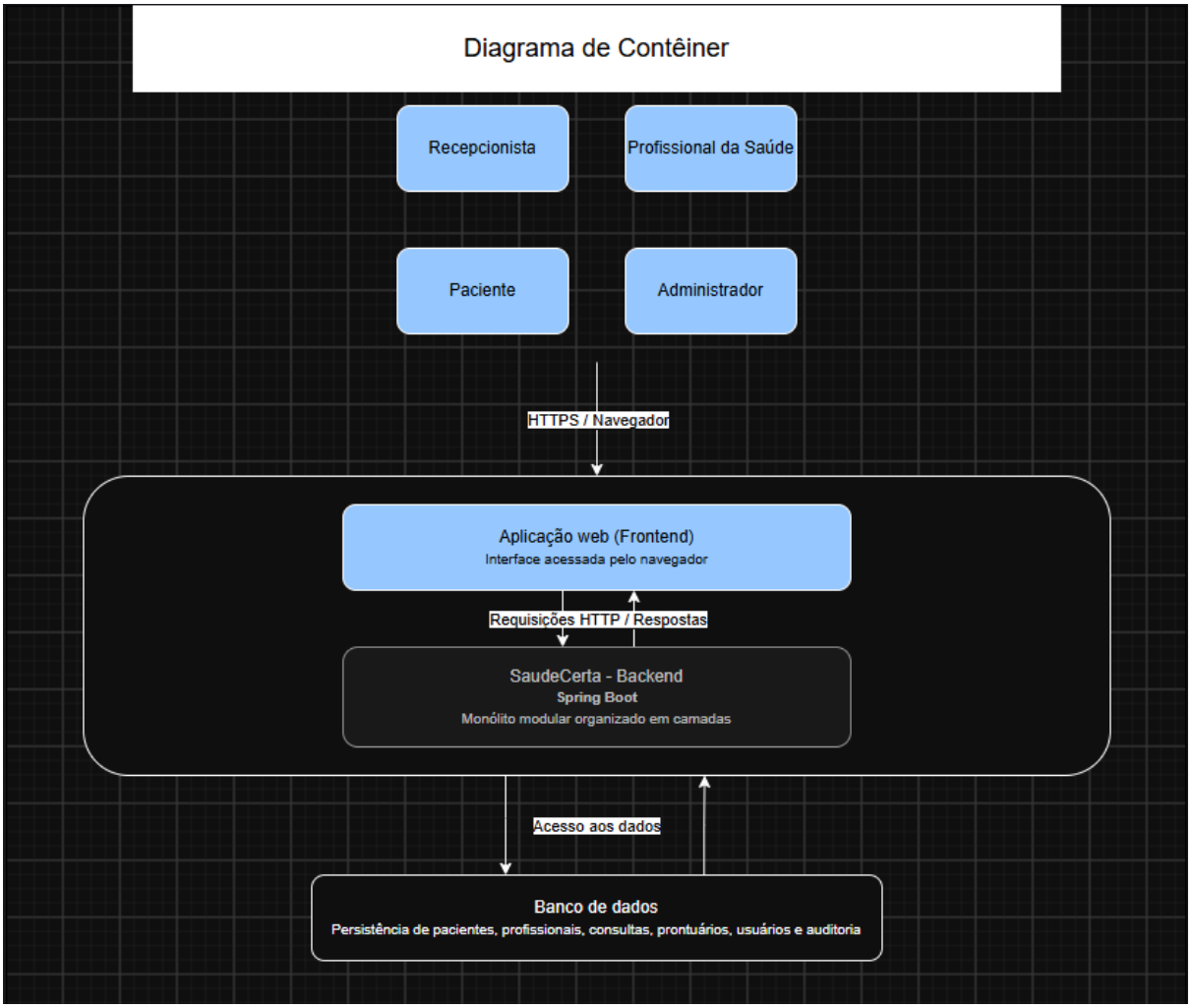
O Diagrama de Contexto apresenta o SaúdeCerta como um único sistema e mostra os principais atores que interagem com ele.

Descrição:

- **Recepcionista:** utiliza o sistema para cadastrar pacientes e organizar os agendamentos.
- **Profissional de saúde:** consulta a agenda, acessa os prontuários autorizados e registra atendimentos.
- **Paciente:** consulta somente seus próprios dados permitidos.
- **Administrador:** gerencia usuários, perfis, auditoria e relatórios.
- **Laboratório externo:** aparece apenas como possibilidade de integração futura, pois essa funcionalidade está fora do escopo atual.

O limite do sistema corresponde à aplicação web SaúdeCerta.

1.2 Diagrama C4 – Contêiner



O Diagrama de Contêiner apresenta as principais partes tecnológicas do sistema e como elas se comunicam.A proposta utiliza uma aplicação web, acessada pelo navegador, um backend desenvolvido em Java com Spring Boot e um banco de dados relacional.

Descrição dos contêineres:

1. Navegador Web – Interface

É a parte utilizada pelos usuários para acessar as funcionalidades do SaúdeCerta. Permite realizar cadastros, consultar informações e gerenciar agendamentos.

2. Aplicação SaúdeCerta – Backend

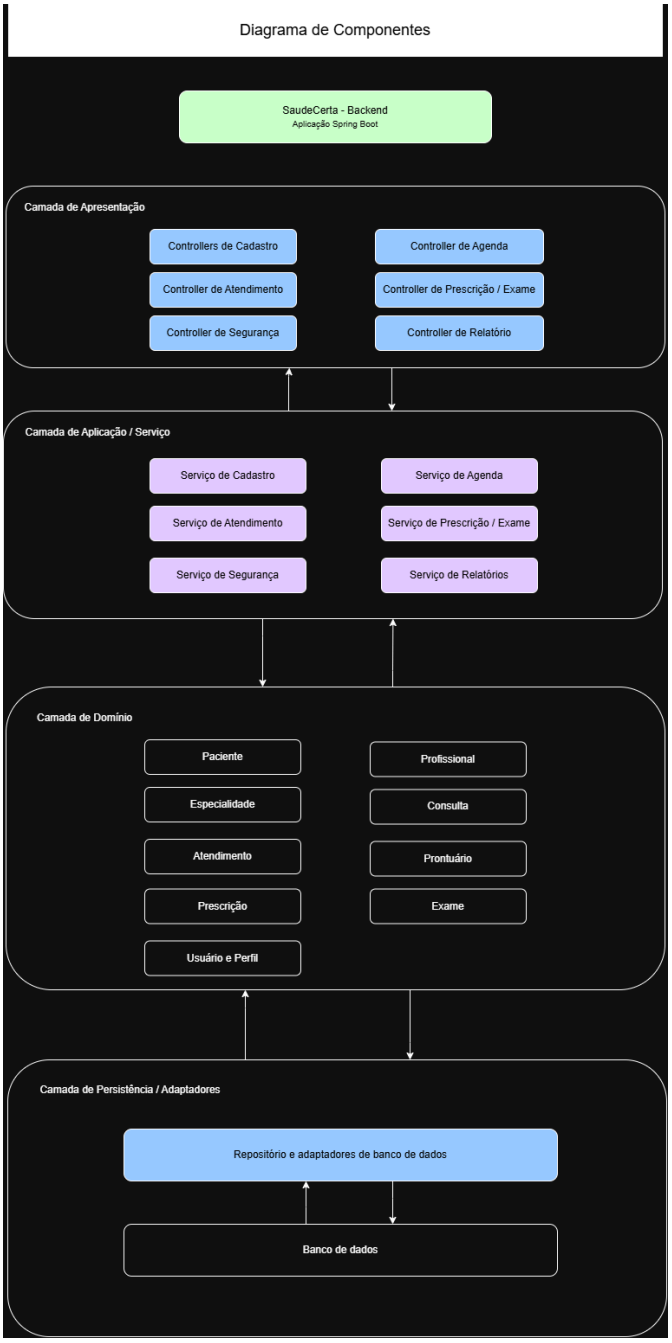
Desenvolvida em Java 17+ com Spring Boot 3.x. É responsável por receber as requisições da interface, executar as regras de negócio e organizar os módulos de cadastro, agenda, prontuário, segurança e auditoria.

3. Banco de Dados Relacional

Responsável por armazenar as informações do sistema, como pacientes, profissionais, consultas, atendimentos, prescrições e exames. O H2 poderá ser utilizado durante o desenvolvimento, enquanto o PostgreSQL é uma opção para implantação.

A comunicação entre o navegador e a aplicação acontece por HTTPS e JSON. O backend utiliza JPA e SQL para acessar o banco de dados.

1.3 Diagrama C4 – Componente



O Diagrama de Componente apresenta a organização interna da aplicação Spring Boot, mostrando os principais componentes e suas responsabilidades.

Descrição dos componentes:

Componente	Responsabilidade
Controllers REST	Receber as requisições HTTP e encaminhá-las aos serviços.
AgendaService	Gerenciar disponibilidade e agendamentos.
AtendimentoService	Registrar atendimentos e atualizar informações do prontuário.

CadastroService	Gerenciar os cadastros de pacientes, profissionais e especialidades.
Domínio	Concentrar as entidades e regras de negócio do sistema.
Repositories	Realizar o acesso aos dados utilizando Spring Data JPA.
Spring Security / RBAC	Controlar autenticação e permissões dos usuários.
AtendimentoAcessado	Evento utilizado para informar que ocorreu um acesso ao prontuário.
AuditoriaService	Processar os eventos e registrar as informações de auditoria.

2. Definição das camadas, componentes e modelo de domínio

2.1 Definição das camadas

A arquitetura do SaúdeCerta será organizada em quatro camadas principais e uma camada transversal, responsável por funcionalidades utilizadas em diferentes partes do sistema.

Camada	Responsabilidade	Componentes e tecnologias
Apresentação	Receber requisições HTTP, validar entradas e apresentar os dados pela API.	Controllers REST, DTOs e Bean Validation.
Aplicação/Serviços	Organizar os casos de uso, executar serviços e controlar transações.	AgendaService, AtendimentoService, CadastroService, @Service e @Transactional.
Domínio	Concentrar as entidades, regras e invariantes de negócio.	Paciente, Consulta, Agenda, Atendimento, Prontuário e regras RN01–RN07.
Persistência/Infraestrutura	Realizar o acesso e o armazenamento dos dados.	Interfaces de repositório, JPA, Spring Data JPA e @Repository.
Transversal	Oferecer funcionalidades comuns de segurança, auditoria, tratamento de erros e configurações.	Spring Security, RBAC, eventos, filtros, logs e @ControllerAdvice.

Camada de Apresentação

É responsável pela comunicação entre o usuário e o sistema. Recebe as requisições, valida os dados de entrada e encaminha as informações para a camada de aplicação.

Camada de Aplicação/Serviços

Responsável por organizar a execução das funcionalidades do sistema. Os serviços coordenam os casos de uso, aplicam as regras necessárias e controlam as transações.

Camada de Domínio

Contém as entidades e as regras de negócio do SaúdeCerta. Essa camada representa os conceitos principais da clínica e não deve depender diretamente do banco de dados ou do framework web.

Camada de Persistência/Infraestrutura

Responsável pelo armazenamento e pela consulta das informações. Utiliza interfaces de repositório e Spring Data JPA para acessar o banco de dados.

Componentes Transversais

São funcionalidades utilizadas por diferentes módulos do sistema. A segurança controla os acessos, a auditoria registra as ações realizadas e o tratamento de erros padroniza as respostas da aplicação.

2.2 Organização sugerida de pacotes

A estrutura do projeto poderá ser organizada da seguinte forma:

```
br.edu.ifpa.saudecerta
│
├── agenda
│   ├── api
│   ├── application
│   ├── domain
│   └── infra
│
├── prontuario
│   ├── api
│   ├── application
│   ├── domain
│   └── infra
│
├── cadastro
│   ├── Paciente
│   ├── Profissional
```

```
|   └── Especialidade
|
├── seguranca
|   ├── Usuario
|   ├── perfis
|   └── auditoria
|
└── shared
    ├── configuracoes
    ├── excecoes
    └── utilitarios
```

As dependências devem apontar para o domínio. A persistência implementa interfaces de repositório, evitando que as regras de negócio dependam diretamente do JPA. Os Controllers não devem concentrar as regras de negócio.

2.3 Padrões e práticas arquiteturais

- **Layered Architecture:** separa o sistema em camadas com responsabilidades diferentes.
- **Ports and Adapters:** mantém o domínio independente dos detalhes externos, como banco de dados e interface.
- **Repository:** organiza o acesso aos dados.
- **Service:** coordena a execução dos casos de uso.
- **DTO:** controla os dados recebidos e enviados pela API.
- **RBAC:** controla as permissões de acordo com o perfil do usuário.
- **Observer/Eventos:** permite publicar eventos, como AtendimentoAcessado, para processamento da auditoria.

2.4 Modelo de domínio

O modelo de domínio representa as principais entidades do SaúdeCerta, seus atributos e relacionamentos.

Entidade	Atributos principais	Relacionamentos
Paciente	id, nome, cpf, dataNascimento, telefone, email	Um paciente possui várias consultas e um prontuário.
Profissional	id, nome, registroConselho, ativo	Possui várias agendas e relacionamento N com especialidades.
Especialidade	id, nome	Possui relacionamento N com profissionais.

Agenda/Disponibilidade	id, diaSemana, horaInicio, horaFim	Várias agendas pertencem a um profissional.
Consulta	id, dataHora, status	Relaciona-se com um paciente, um profissional e pode possuir um atendimento.
Atendimento	id, dataHora, queixa, evolucao, conduta	Está ligado a uma consulta e pode possuir várias prescrições e solicitações de exames.
Prontuário	id, pacienteId	Possui relacionamento 1:1 com paciente e reúne os atendimentos.
Prescrição	id, dataEmissao	Está ligada a um atendimento e possui vários itens.
ItemDePrescrição	id, medicamento, dose, posologia	Vários itens pertencem a uma prescrição.
SolicitaçãoDeExame	id, tipoExame, status	Várias solicitações pertencem a um atendimento.
Usuário	id, login, senhaHash, perfil	Pode estar associado a um profissional ou paciente.

Regras de domínio

As seguintes regras devem ser respeitadas pelo sistema:

- **RN01:** o CPF deve ser único entre os pacientes ativos.
- **RN02:** um profissional não pode ter duas consultas com status AGENDADA no mesmo horário.
- **RN03:** o atendimento somente pode ser registrado para uma consulta cuja data seja hoje ou anterior.
- **RN04:** prescrições e solicitações de exames devem estar vinculadas a um atendimento.
- **RN05:** o prontuário deve ser append-only, ou seja, os atendimentos não podem ser apagados, apenas complementados.
- **RN06:** o paciente acessa somente os próprios dados, e a recepção não pode acessar o conteúdo clínico.
- **RN07:** uma consulta somente pode ser cancelada quando estiver com status AGENDADA.

3. Cenários de qualidade para os requisitos não funcionais

Os cenários de qualidade descrevem como o sistema deve se comportar diante de determinadas situações. Eles apresentam a fonte do estímulo, o ambiente, a resposta esperada e uma medida que permite verificar se o requisito foi atendido.

3.1 Cenários de qualidade

Requisito	Fonte e estímulo	Ambiente	Resposta esperada	Medida de sucesso
RNF01 – Segurança	Usuário sem permissão tenta acessar um dado clínico.	Sistema em funcionamento normal.	Negar o acesso e registrar a tentativa.	100% das tentativas não autorizadas devem ser bloqueadas.
RNF02 – Auditabilidade	Profissional abre um prontuário.	Usuário autenticado e autorizado.	Registrar usuário, paciente, data, horário e ação, permitindo consulta administrativa.	Registrar cada acesso e permitir consulta em até 1 minuto.
RNF03 – Desempenho	Recepcionista pesquisa um paciente pelo nome.	Carga normal da base de dados.	Retornar os resultados da busca rapidamente.	Menos de 2 segundos no percentil 95 (p95).
RNF04 – Disponibilidade	Usuário solicita acesso à agenda.	Durante as 12 horas diárias de funcionamento.	Manter a agenda acessível.	Disponibilidade mensal igual ou superior a 99% nesse período.
RNF05 – Modificabilidade	Equipe adiciona o módulo de faturamento no futuro.	Durante a evolução planejada do sistema.	Adicionar o módulo sem alterar o núcleo da agenda e do prontuário.	Esforço de até 5 dias-equipe.
RNF06 – Usabilidade	Recepcionista realiza um agendamento simples.	Uso normal, a partir da tela inicial.	Concluir o agendamento sem ajuda externa.	Até 1 minuto e no máximo 5 cliques.
RNF07 – Segurança de credenciais	Sistema armazena e verifica uma senha.	Durante cadastro e autenticação.	Armazenar hash com salt, nunca a senha em texto puro.	Utilizar BCrypt ou equivalente e verificar em revisão de código.

3.2 Táticas arquiteturais associadas

Para atender aos requisitos não funcionais, serão utilizadas as seguintes táticas:

Segurança: autenticação, controle de acesso por perfis (RBAC), princípio do menor privilégio, armazenamento de senhas com hash e auditoria.

Desempenho: utilização de índices no banco de dados, paginação, consultas específicas e prevenção de consultas N+1.

Disponibilidade: tratamento de exceções, timeouts, transações curtas e healthcheck para verificar o funcionamento da aplicação.

Modificabilidade: módulos com alta coesão, baixo acoplamento e interfaces bem definidas.

Auditabilidade: utilização de eventos de domínio e logs estruturados e protegidos contra alterações indevidas.

4. Registros de Decisões Arquiteturais (ADRs)

Os ADRs (Architecture Decision Records) são registros que documentam as decisões tomadas durante a definição da arquitetura. Eles apresentam o contexto, as alternativas consideradas, a decisão escolhida e suas consequências.

Foram definidos quatro ADRs para o SaúdeCerta.

ADR-001 – Adoção de monólito modular inicialmente

Status: Proposto.

Contexto:

A clínica possui porte médio e o projeto precisa organizar diferentes funcionalidades sem aumentar desnecessariamente a complexidade de desenvolvimento e implantação.

Alternativas consideradas:

1. **Microsserviços:** cada funcionalidade seria desenvolvida e executada como um serviço independente.
2. **Aplicação única sem separação modular:** todas as funcionalidades ficariam concentradas em uma estrutura sem divisão clara.
3. **Monólito modular:** uma única aplicação organizada em módulos com responsabilidades bem definidas.

Decisão:

Adotar inicialmente o monólito modular utilizando Spring Boot, organizando o sistema em módulos de cadastro, agenda, prontuário, segurança e auditoria.

Trade-offs:

A escolha reduz a complexidade de implantação, comunicação de rede e monitoramento. Entretanto, exige disciplina na organização do código para evitar dependências excessivas entre os módulos.

Os microsserviços permitiriam escalar cada funcionalidade separadamente, mas aumentariam os custos de implantação, testes e monitoramento.

Análise de custo/benefício:

O custo inicial é menor porque a aplicação será implantada como uma única unidade. O benefício é facilitar o desenvolvimento e a manutenção, mantendo a possibilidade de separar futuramente módulos como Agendamento ou Prontuário, caso exista necessidade.

ADR-002 – Auditoria por eventos assíncronos

Status: Proposto.

Contexto:

O requisito RNF02 determina que os acessos aos prontuários sejam registrados. Ao mesmo tempo, o sistema precisa manter um bom desempenho durante as consultas.

Alternativas consideradas:

1. **Auditoria síncrona:** registrar a auditoria imediatamente durante a mesma operação de acesso.
2. **Auditoria assíncrona:** publicar um evento e realizar o processamento do registro separadamente.
3. **Não registrar cada acesso:** não manter um histórico completo das consultas aos prontuários.

Decisão:

Adotar auditoria por eventos assíncronos. Quando ocorrer um acesso ao prontuário, o sistema poderá publicar um evento de domínio, como AtendimentoAcessado, que será processado pelo componente AuditoriaService.

Trade-offs:

O processamento assíncrono reduz o trabalho no fluxo principal e diminui o acoplamento entre o atendimento e a auditoria. Entretanto, pode ocorrer um pequeno atraso no registro e será necessário tratar falhas e permitir o reprocessamento dos eventos.

A auditoria síncrona facilita a confirmação imediata do registro, mas pode aumentar o tempo de resposta.

Análise de custo/benefício:

O custo está na implementação do mecanismo de eventos e no tratamento de falhas. O benefício é manter o desempenho do sistema e organizar a auditoria de forma independente. Como os dados são sensíveis, devem existir mecanismos para evitar a perda dos registros.

ADR-003 – Controle de acesso com RBAC e Spring Security

Status: Proposto.

Contexto:

O SaúdeCerta possui diferentes tipos de usuários, como pacientes, recepcionistas, profissionais de saúde e administradores. Cada perfil precisa ter permissões específicas.

Alternativas consideradas:

1. **Verificações manuais:** cada Controller seria responsável por verificar as permissões.
2. **Acesso igual para todos:** todos os usuários teriam acesso às mesmas funcionalidades.
3. **RBAC com Spring Security:** as permissões seriam organizadas de acordo com os perfis dos usuários.

Decisão:

Adotar o controle de acesso baseado em perfis (RBAC), utilizando Spring Security para autenticação e autorização. Também será verificado se o paciente está acessando somente seus próprios dados.

As senhas serão armazenadas com hash e salt, utilizando BCrypt ou mecanismo equivalente.

Trade-offs:

A centralização das permissões reduz o risco de esquecer verificações de segurança em alguma funcionalidade. Porém, exige configuração adequada e testes para cada perfil e recurso.

Análise de custo/benefício:

O custo está relacionado à configuração do Spring Security e à definição das permissões. O benefício é proteger as informações clínicas, reduzir acessos indevidos e facilitar o gerenciamento dos usuários.

ADR-004 – Banco de dados relacional com Repository e JPA

Status: Proposto.

Contexto:

O sistema possui várias entidades relacionadas, como pacientes, consultas, profissionais, atendimentos, prescrições e exames. É necessário escolher uma forma organizada de armazenar e consultar essas informações.

Alternativas consideradas:

1. **SQL diretamente nos serviços:** as consultas ao banco ficariam misturadas com as regras de negócio.

2. **Banco NoSQL:** utilizar uma estrutura não relacional para armazenar as informações.
3. **Banco relacional com Repository e JPA:** organizar os dados em tabelas relacionadas e separar o acesso ao banco por meio de repositórios.

Decisão:

Adotar um banco de dados relacional, utilizando Spring Data JPA e o padrão Repository. O H2 será utilizado no desenvolvimento e o PostgreSQL poderá ser utilizado na implantação.

Trade-offs:

A solução favorece a integridade dos dados e facilita consultas entre entidades relacionadas. Entretanto, exige um mapeamento correto dos relacionamentos e atenção ao desempenho das consultas, principalmente em situações de N+1 e controle de transações.

Análise de custo/benefício:

O custo envolve configurar o banco de dados, mapear as entidades e manter os repositórios. O benefício é organizar as informações, manter os relacionamentos entre os dados e separar as regras de negócio dos detalhes de persistência.

5. Matriz de rastreabilidade requisitos × arquitetura

A matriz de rastreabilidade apresenta a relação entre os requisitos levantados na primeira entrega e os elementos da arquitetura definidos nesta segunda etapa.

Ela permite identificar quais decisões, componentes e regras serão responsáveis por atender cada requisito e como será possível verificar seu funcionamento.

5.1 Rastreabilidade dos requisitos funcionais

Requisito	Decisão ou regra relacionada	Componente ou tática	Como verificar
RF01 – Pacientes	ADR-004; RN01	PacienteService e PacienteRepository; CPF único.	Cadastrar um paciente válido e rejeitar CPF duplicado entre pacientes ativos.
RF02 – Profissionais e especialidades	ADR-004	CadastroService; associação Profissional–Especialidade.	Testar o cadastro e a associação entre profissionais e especialidades.
RF03 – Disponibilidade	ADR-001 e ADR-004	AgendaService e AgendaRepository.	Consultar horários cadastrados e disponíveis.

RF04 – Agendar consulta	ADR-001; RN02	AgendaService.validarConflito().	Agendar um horário livre e tentar repetir o profissional e horário.
RF05 – Remarcar e cancelar	RN07	ConsultaService; validação de status.	Testar alteração de horário e rejeitar cancelamento fora do status AGENDADA.
RF06 – Impedir conflitos	RN02	Regra de domínio e validação transacional.	Testar duas consultas para o mesmo profissional no mesmo horário.
RF07 – Registrar atendimento	RN03 e RN05	AtendimentoService e entidade Atendimento.	Testar data permitida e preservação do histórico.
RF08 – Prescrições	RN04	PrescricaoService e ItemDePrescrição.	Rejeitar prescrição sem atendimento vinculado.
RF09 – Exames	RN04	SolicitacaoExameService.	Rejeitar solicitação de exame sem atendimento vinculado.
RF10 – Histórico	ADR-004; RN05	ProntuarioService e ProntuarioRepository.	Consultar o histórico e verificar que os atendimentos não podem ser excluídos.
RF11 – Autenticação e perfis	ADR-003	Spring Security, Usuario e RBAC.	Testar login e permissões de cada perfil.
RF12 – Auditoria de acesso	ADR-002 e ADR-003	AtendimentoAcessado e AuditoriaService.	Abrir um prontuário e conferir usuário, paciente, horário e ação registrada.
RF13 – Relatórios	ADR-004	RelatorioService e consultas específicas.	Comparar os resultados dos relatórios com os registros de teste.

5.2 Rastreabilidade dos requisitos não funcionais

Requisito	Decisão/ADR	Componente ou tática	Como verificar
RNF01 – Segurança e privacidade	ADR-003	Spring Security, RBAC e princípio do menor privilégio.	Realizar testes de acesso por perfil e confirmar o bloqueio e registro das tentativas não autorizadas.
RNF02 – Auditabilidade	ADR-002	Evento AtendimentoAcessado e AuditoriaService.	Conferir o registro completo e medir se a consulta administrativa ocorre em até 1 minuto.
RNF03 – Desempenho	ADR-004	Índices, paginação, consultas específicas e prevenção de N+1.	Realizar teste de carga e verificar se o p95 fica abaixo de 2 segundos.
RNF04 – Disponibilidade	ADR-001	Healthcheck, timeouts e tratamento de falhas.	Monitorar a disponibilidade mensal durante as 12 horas diárias de funcionamento.
RNF05 – Modificabilidade	ADR-001 e arquitetura em camadas	Módulos coesos, interfaces/ports e baixo acoplamento.	Adicionar um módulo futuro sem alterar o núcleo da agenda e do prontuário, medindo o esforço.
RNF06 – Usabilidade	Fluxo de interface	Tela de agendamento, busca e validações claras.	Testar com uma recepcionista: concluir em até 1 minuto e no máximo 5 cliques.
RNF07 – Segurança de credenciais	ADR-003	Hash com salt, como BCrypt.	Inspecionar o armazenamento e revisar o código para confirmar que não há senhas em texto puro.

5.3 Relação entre ADRs e requisitos

ADR	Decisão arquitetural	Requisitos relacionados
ADR-001	Monólito modular	RF03, RF04, RNF04 e RNF05.
ADR-002	Auditoria por eventos assíncronos	RF12 e RNF02.

ADR-003	Controle de acesso com RBAC e Spring Security	RF11, RF12, RNF01 e RNF07.
ADR-004	Banco relacional, Repository e JPA	RF01, RF02, RF03, RF07, RF10, RF13 e RNF03.